객체지향언어인 JAVA

객체지향언어와 JAVA

JAVA

객체지향언어로써 객체지향의
특징을 가지고 있는
프로그래밍 언어

객체지향언어와 JAVA

객체지향 프로그래밍

기존의 절차지향 프로그래밍 방식이 아닌 객체의 개념을 이용해서 객체를 중심으로 프로그래밍 하는 것

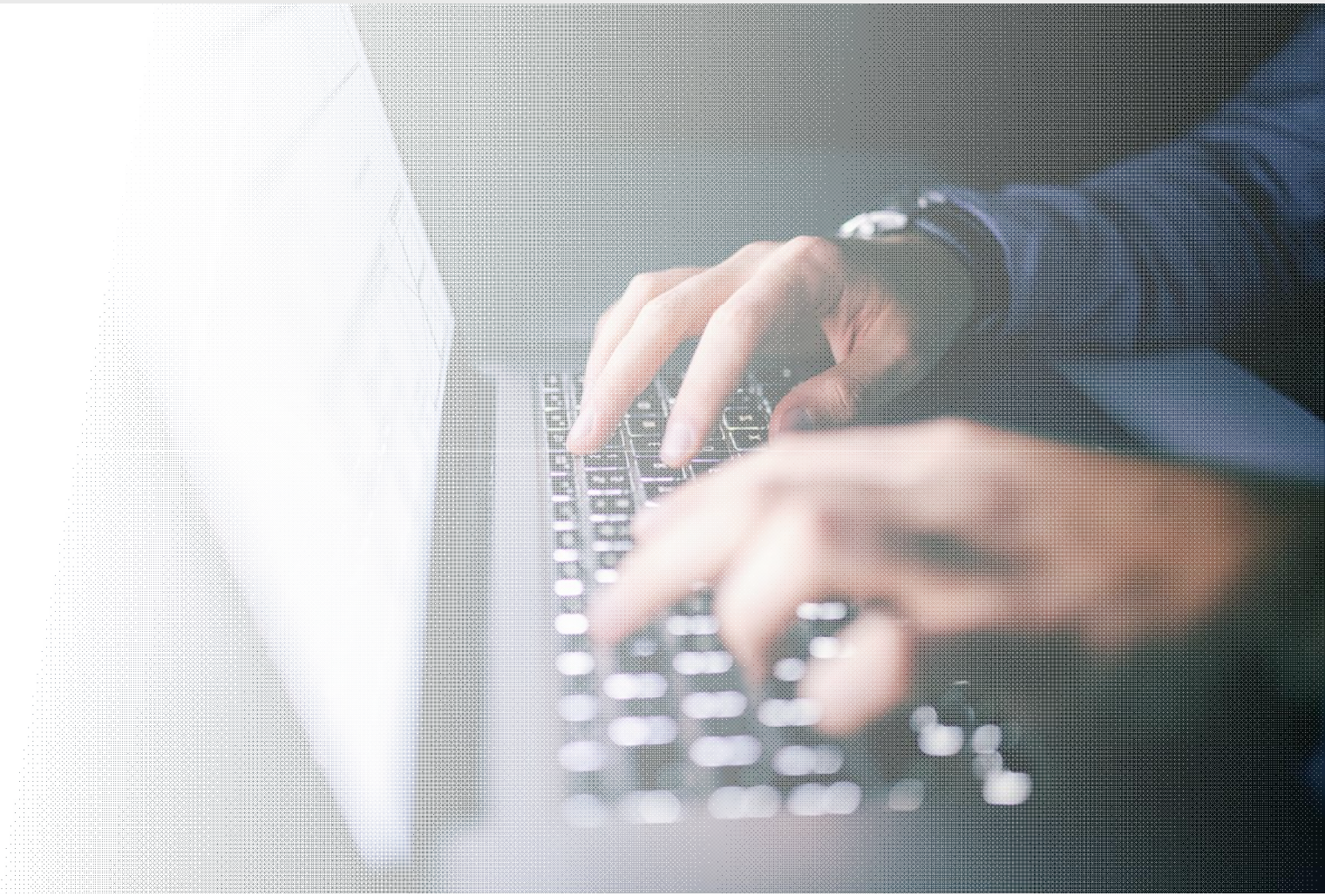
객체들을 속성과 기능으로 정의하고 객체 간의 관계를 실제의 흐름 그대로
프로그래밍에 반영하여 프로그래밍 하는 것

용어	의미
모듈	프로그램을 구성하는 구성요소로 관련된 데이터와 함수를 묶어 사용할 때 이 묶음을 모듈이라고 함 일반적으로 파일로 분리해서 생성하고, JAVA에서는 모듈의 단위를 클래스로 정의해서 사용함
라이브러리	코드의 재사용을 목적으로 서로 관련되고 자주 사용되는 기능을 구현하고 있는 모듈들을 모아 제공하는 것
프레임워크	애플리케이션의 틀과 구조를 이미 정의하고 그 틀(구조) 안에서 프로그래밍이 가능하도록 기능을 제공하는 라이브러리들의 집합 프레임워크를 이용하면 복잡한 구조의 설계를 하지 않아 개발 시간을 단축시킬 수 있음

객체지향언어와 JAVA

객체지향 프로그래밍 과정

- 요구사항 분석
- 객체 추출
- 객체 모델링
- 클래스 정의
- 객체 생성과 활용



객체지향언어와 JAVA

인스턴스와 클래스

JAVA는 객체지향언어로써 객체지향의 특징을 가지고 있는 프로그래밍 언어

객체를 정의할 때 속성은 변수로 기능은 메소드로 만들고 변수들과 메소드들을 하나의 타입으로 묶어 사용

클래스(class)	객체를 만들기 위해 변수와 메소드들로 만들어지는 묶음, 즉 설계도와 같은 것

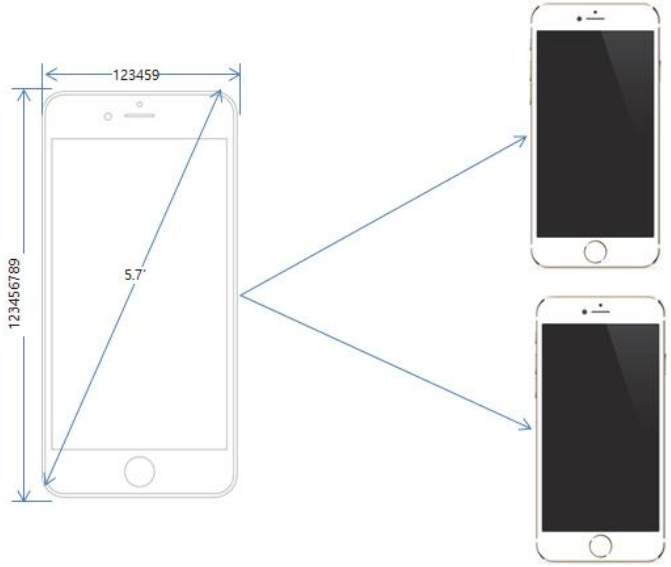
클래스는 객체를 만들기 위해 객체와 연관된 변수와 메소드를 정의해 놓은 설계도

클래스는 설계도이기 때문에 당장 사용할 수 없고, 이 설계도를 기반으로 객체를 생성해서 사용해야 함

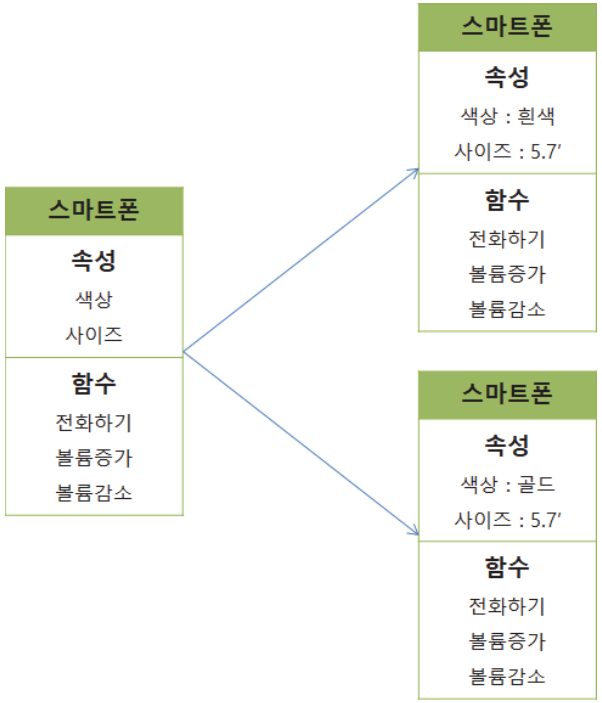
객체지향언어와 JAVA

인스턴스와 클래스

객체를 만들기 위해 변수와 메소드를 정하는 클래스를 만들고, 이 클래스를 이용해서 객체를 생성하고
생성된 객체의 변수와 메소드를 사용하여 프로그래밍



<객체의 개념과 설계도>



<JAVA에서의 객체와 클래스>

클래스 구성과 인스턴스 생성

클래스의 구조

JAVA에서 클래스를 정의하는 문법 구조

클래스를 정의하는 키워드

클래스 정의 블록

```
class 클래스 명
{
    데이터 타입 변수명;
    생성자(){}
    반환 데이터 타입 메서드명{

    }
}
```

클래스의 멤버 변수

클래스의 멤버 메서드

클래스의 구조

클래스 정의하기

1 키워드인 class를 기술하고 클래스임을 선언

2 class 키워드 뒤에는 클래스 이름을 기술

- 클래스 이름은 대문자로 시작하는 영문이름으로 작성
- 소문자로 작성을 해도 가능하지만 암묵적인 약속으로 대문자로 시작

대문자로 시작하는 영문 이름을 쓰기로 한 이유

우리가 사용하는 식별자(변수의 이름, 메소드의 이름, 클래스의 이름)들 간의 구별을 위한 것

클래스의 구조

클래스 정의하기

3 중괄호 블록 “{}” 안에 클래스의 멤버들을 작성

- 속성을 표현하는 변수들과 기능을 표현하는 메소드들 그리고 생성자가 있음
- 필요한 속성들을 변수로 선언하여 정의
  - + 이 변수를 멤버 변수 또는 인스턴스 변수라고 함
- 행위 또는 기능의 표현은 메소드 형식으로 정의
- 메소드의 구성은 기능 실행 후 반환할 데이터의 타입을 정의해주고 메소드의 이름을 기술
- 메소드 이름 뒤에는 “()” 괄호 블록으로 필요한 데이터를 어떤 타입으로 몇 개를 받을지 정의

클래스의 구조

스마트폰을 정의하는 클래스 정의하기

```
class SmartPhone {

  String color = "white"; // 속성 : 스마트폰의 색상
  float size = 5.7F;  // 속성 : 스마트폰의 사이즈
  int volume = 0; // 속성 : 음량 크기

  void call() { // 기능 : 전화걸기
   System.out.println("전화걸기");
  }
  void volumeUp() { // 기능 : 음량  키우기
   volume += 1;
  }
  void volumeDown() { // 기능 : 음량 낮추기
    volume -= 1;
  }
}
```

속성, 변수

기능, 메소드

클래스의 구조

스마트폰을 정의하는 클래스 정의하기

```
package chapter05;

public class SmartPhone {

	String color = "white"; // 속성 : 스마트폰의 색상
	float size = 5.7F; // 속성 : 스마트폰의 사이즈
	int volume = 0; // 속성 : 음량 크기

	void call() { // 기능 : 전화걸기
		System.out.println("전화걸기");
	}
```

클래스의 구조

스마트폰을 정의하는 클래스 정의하기

```
        void volumeUp() { // 기능 : 음량 키우기
                volume += 1;
        }

        void volumeDown() { // 기능 : 음량 낮추기
                volume -= 1;
        }
}
```

인스턴스의 생성

클래스를 정의하는 것만으로는 프로그램에서 사용할 수 없음

프로그래밍을 위해서는 새롭게 정의한 클래스를 이용해서 인스턴스를 생성해야만 함

인스턴스 생성	클래스에 정의된 변수와 메소드를 메모리에 로드해서 변수를 참조하고 메소드를 호출할 수 있는 상태로 만드는 것

! 인스턴스를 만드는 과정을 '인스턴스화' 라고 함

인스턴스의 생성

인스턴스 생성방법

1. 인스턴스의 메모리 주소를 저장하기 위해 참조변수를 선언
2. 인스턴스를 생성해서 생성된 인스턴스의 주소 값을 참조변수에 저장
3. 객체의 생성은 new 키워드를 사용 해서 생성
4. new 키워드 뒤에 클래스이름과 괄호"()"를 붙여 생성

```
클래스이름 참조변수이름 = new 클래스이름();
```

SmartPhone 클래스의 인스턴스를 생성

```
SmartPhone sp = new SmartPhone();
```

인스턴스의 생성

인스턴스 생성과정

```
SmartPhone sp = new SmartPhone();
```

참조변수 sp 생성

0×100

객체 생성 → 메모리주소 0×100

color
size
volume
call()
volumeUp()
volumeDown()

인스턴스의 생성

참조변수의 예

```
SmartPhone sp1 = new SmartPhone();
SmartPhone sp2 = sp1;
```

참조변수 sp1

인스턴스 생성을 통해 참조 값을 할당 받았음

참조변수 sp2

참조변수 sp1이 가지고 있는 주소 값을 대입

참조변수는 인스턴스 자체를 저장하는 것이 아니라 인스턴스의 주소 값을 가지는 것임을 구분해야 함

인스턴스의 멤버에 접근

생성된 인스턴스의 멤버인 변수를 참조하거나 메소드를 호출하는 방법

참조변수가 가지는 주소 값을 통해 해당 인스턴스의 메모리에 접근하고, 인스턴스 안에서 변수를 참조하여, 메소드를 호출

인스턴스의 변수 참조

참조변수.변수이름

```
SmartPhone sp = new SmartPhone();
System.out.println(sp.color);
sp.color = "red";
```

인스턴스의 메소드 호출

참조변수.메소드이름()

```
SmartPhone sp = new SmartPhone();
sp.call();
sp.volumeUp();
sp.volumeDown();
```

인스턴스의 멤버에 접근

```
package chapter05;

public class SmartPhoneMain {

    public static void main(String[] args) {

        SmartPhone sp = new SmartPhone();  // SmartPhone 클래스로 객체 생성

        System.out.println("> color : " + sp.color);  // 참조변수를 이용해 객체의 color변수 참조
        System.out.println("> size : " + sp.size);  // 참조변수를 이용해 객체의 color변수 참조
        System.out.println("> volume : " + sp.volume); // 참조변수를 이용해 객체의 color변수 참조

        sp.call();  // 참조변수를 이용해 객체의 메소드 호출
    }
}
```

클래스 선언과 여러 개의 객체 생성

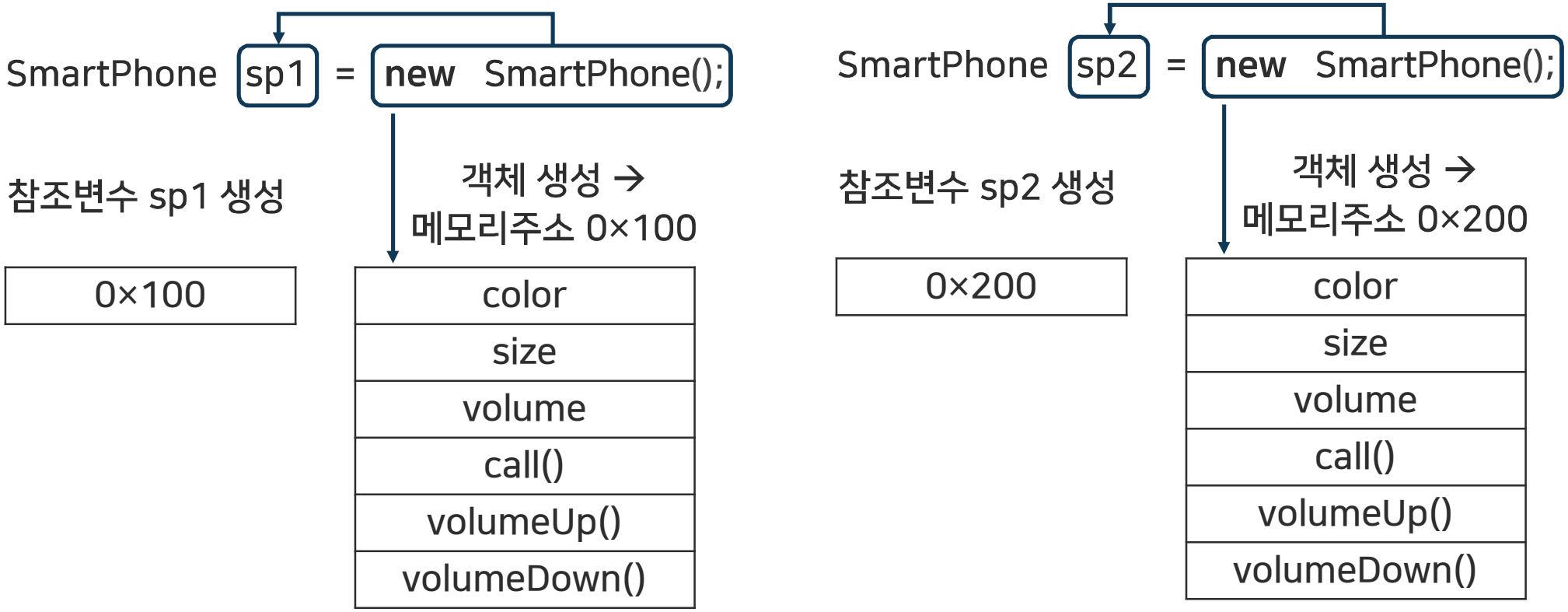
```
SmartPhone sp1 = new SmartPhone();
SmartPhone sp2 = new SmartPhone();
```

참조변수 sp1과 sp2는 각각 다른 인스턴스를 생성하고, 서로 다른 인스턴스를 가리키는 주소 값을 가지고 있음

인스턴스를 생성하면서 서로 다른 메모리 공간을 생성하고 있는 것

서로 다른 독립적인 인스턴스를 생성한 것이고, 각각 독립적인 변수가 메모리에 생성된 것

클래스 선언과 여러 개의 객체 생성



클래스 선언과 여러 개의 객체 생성

```
package chapter05;

public class MultiInstance {

    public static void main(String[] args) {

        SmartPhone sp1 = new SmartPhone(); // SmartPhone 클래스로 객체 생성
        SmartPhone sp2 = new SmartPhone(); // SmartPhone 클래스로 객체 생성

        sp1.color = "Gold"; // 첫 번째 객체의 변수 color 속성값 대입
        sp1.size = 5.7F; // 첫 번째 객체의 변수 size 속성값 대입
        sp1.volume = 5; // 첫 번째 객체의 변수 volume 속성값 대입
        System.out.println("> color : " + sp1.color); // 첫 번째 객체의 변수들을 출력
        System.out.println("> size : " + sp1.size); //
        System.out.println("> volume : " + sp1.volume); //
```

클래스 선언과 여러 개의 객체 생성

```
        sp2.color = "Black"; // 두 번째 객체의 color 속성값 대입
        sp2.size = 5.7F; // 두 번째 객체의 size 속성값 대입
        sp2.volume = 3; // 두 번째 객체의 volume 속성값 대입
        System.out.println("> color : " + sp2.color); // 두 번째 객체의 변수들을 출력
        System.out.println("> size : " + sp2.size); //
        System.out.println("> volume : " + sp2.volume); //

        sp1.volumeUp(); // 첫 번째 객체의 volumeUp() 메서드 호출 : volume 5 -> 6
        System.out.println("SmartPhone 1 객체의 volumeUp() 메서드를 호출했습니다.");
        System.out.println("> volume : " + sp1.volume); // 5가 아닌 6이 출력

        sp2.volumeDown(); // 첫번재 객체의 volumeDown() 메서드 호출
                          // volume 3 -> 2
        System.out.println("SmartPhone 2 객체의 volumeDown() 메서드를 호출했습니다.");
        System.out.println("> volume : " + sp2.volume); // 3이 아닌 2가 출력
    }
}
```

메소드의 오버로딩과 클래스 멤버의 구분

메소드의 오버로딩

JAVA에서는 기본적으로 동일한 이름의 메소드 정의는 불가하지만 특정 조건을 만족할 때
동일한 이름의 메소드를 정의할 수 있음

같은 이름의 메소드를 정의하는 것을 메소드의 오버로딩(overloading)이라고 함

같은 이름의 메소드 정의	매개변수의 타입을 다르게 하거나, 매개변수의 개수를 다르게 정의

오버로딩 된
메소드의 호출

메소드 호출 시에 전달하는 매개변수의 개수 호출 또는 타입이 일치하는 메소드 호출

메소드의 오버로딩

println() 메소드

JAVA API에서 제공하는 클래스에서 오버로딩으로 정의되어 있는 대표적인 메소드

+ println(): 출력 없이 개행
+ println(boolean x): 논리값을 출력
+ println(char x): char타입의 데이터를 받아 출력
+ println(char[] x): char타입의 배열을 전달받아 출력
+ println(double x): double타입의 데이터를 전달받아 출력
+ println(float x): float타입의 데이터를 전달받아 출력
+ println(int x): int타입의 데이터를 전달받아 출력
+ println(long x): long타입의 데이터를 전달받아 출력
+ println(Object x): Object타입의 객체를 전달받아 출력
+ println(String x): String타입의 데이터를 전달받아 출력

메소드의 오버로딩

오버로딩 메소드 정의

```
int add(int n1, int n2) {
    return n1 + n2;
}

float add(float n1, float n2) {
    return n1 + n2;
}

void add(int n1, int n2) {
    System.out.println(n1+n2)
}

int add(int num1, int num2) {
    return n1 + n2;
}
```

두 메소드는 매개변수의 타입이 다르기 때문에 오버로딩이 성립됩니다.

이 메소드는 매개변수는 같고 메소드의 반환 타입이 다른 형태로 정의 되어 오버로딩이 성립되지 않습니다.

이 메소드는 매개변수의 이름이 다르게 정의 되어 있으나 호출 시에는 변수의 이름으로 구분이 되지 않아 오버로딩이 성립되지 않습니다.

메소드의 오버로딩

오버로딩 메소드의 호출

```
add(10, 20)        매개변수의 인자가 모두 int 타입이므로 int add(int n1, int n2)
                       호출
add(10.1f, 20.2f)      매개변수의 인자가 모두 float 타입이므로 float add(float
                           n1,float n2)
```

메소드의 오버로딩

```
package chapter05;

public class Adder {

        int add(int n1, int n2) { //오버로딩
            return n1 + n2;
        }

        float add(float n1,float n2) { //오버로딩
            return n1 + n2;
        }
}
```

메소드의 오버로딩

```
package chapter05;

public class AdderMain {

	public static void main(String[] args) {

		Adder adder = new Adder(); //객체 생성

		System.out.println(adder.add(10, 20)); // add(int n1, int n2) 호출
		System.out.println(adder.add(10.1f, 20.2f)); // add(float n1,float n2)호출
	}
}
```

클래스 멤버의 구분

변수의 구분

변수는 선언된 위치에 따라 변수의 생명주기가 달라짐

선언되는 위치에 따라 지역 변수, 인스턴스 변수, 클래스 변수로 나누어짐

지역 변수	메소드의 블록 안에서 선언되는 변수, 매개변수
인스턴스 변수	클래스 블록 안에서 선언되는 변수, 매개변수
클래스 변수	인스턴스 변수 앞에 static 키워드가 있는 변수

구분	생성시기	소멸시기
지역 변수	변수의 선언 명령문이 실행될 때	해당 변수를 감싸고 있는 블록이 종료될 때
인스턴스 변수	객체가 생성될 때	객체가 소멸될 때
클래스 변수	프로그램이 실행될 때	프로그램이 종료될 때 소멸

클래스 멤버의 구분

변수의 구분

1 지역 변수

- 지역 변수는 메소드의 블록 안에서 선언되는 변수
- 메소드의 선언 부에 정의하는 매개변수 변수도 지역 변수의 특징을 가지고 있음
- 지역 변수는 변수의 메모리가 생성되는 시점은 변수의 선언문이 실행될 때 변수의 메모리가 생성되고, 이 메모리 영역은 생성된 메소드가 종료되는 시점에 소멸
- 조건문, 반복문의 “{}”괄호 블록 내에서 선언될 수 있고 블록의 끝에 오면 변수에 할당된 메모리 공간이 소멸됨
- 메소드 내에서 정의된 지역 변수는 메소드 블록 내에서만 사용이 가능하고 메소드를 벗어난 영역에서는 해당 변수를 참조할 수 없음
- 특정 기능을 수행할 때만 필요한 변수를 사용해야 할 경우 지역 변수로 선언해서 사용

클래스 멤버의 구분

변수의 구분

1 지역 변수

```
package chapter05;

public class LocalVariable {
    void method(int num) { // 매개변수도 지역 변수

        int lv = 10; // 메소드 블록 안에서 정의하는 변수는 지역 변수

        if (num > 10) {
            boolean check = true; // 블록 안에 정의되는 변수는 블록 안에서만 사용 가능한
                                  지역 변수
        } else {
            boolean check = false;
        }
```

클래스 멤버의 구분

변수의 구분

1 지역 변수

```
        for (int i = 0; i < 5; i++) { // for 정의 부에서 선언된 메소드는 for 블록 내부에서만 사용
                                        가능한 지역 변수
            System.out.println(i);
        }

        while (lv < 0) {
            int lv2 = 0; // while 블록 안에 정의되는 변수는 while 블록 안에서만 사용 가능한
                            지역 변수
            System.out.println(lv2);
        }
    }
}
```

클래스 멤버의 구분

변수의 구분

2 인스턴스 변수

- 인스턴스 변수는 클래스의 멤버가 정의되는 영역에 선언되는 변수
- 인스턴스가 생성되는 시점에 변수의 메모리가 생성되고, 해당 인스턴스가 소멸될 때 메모리 공간도 소멸
- 인스턴스의 변수는 인스턴스 멤버들 간에 참조가 가능
- 인스턴스의 멤버로 정의된 메소드에서 같이 공유해서 사용해야 하는 경우 인스턴스 변수로 선언해서 사용
- 인스턴스 변수를 참조하는 방법은 “참조변수.변수이름” 형식으로 참조

참조변수.변수이름

클래스 멤버의 구분

변수의 구분

2 인스턴스 변수

```
package chapter05;

public class InstanceVariable {

        String name = "KING";     // 인스턴스 변수 name 선언
        int age = 12;                          // 인스턴스 변수 age 선언

        void tellName() {
                System.out.println("나의 이름은 "+name+"입니다."
                // 멤버변수인 name을 참조한다.
        }
```

클래스 멤버의 구분

변수의 구분

2 인스턴스 변수

```
        void tellAge() {
                System.out.println("나는 "+age+"살 입니다.");
                // 멤버변수인 age를 참조한다.
        }

        public static void main(String[] args) {

                InstanceVariable variable = new InstanceVariable();
                variable.tellName(); // 인스턴스 메소드 호출
                variable.tellAge(); //인스턴스 메소드 호출
        }
}
```

클래스 멤버의 구분

변수의 구분

3 클래스 변수

- 클래스변수는 선언되는 위치가 인스턴스 변수와 같지만 변수 앞에 static 키워드를 붙여 사용하는 변수
- 클래스변수의 생성 시점은 클래스의 바이트 코드가 메모리에 로드가 될 때 변수의 메모리 공간도 생성이 되고, 프로그램이 종료될 때 클래스변수의 메모리 공간도 함께 소멸
- 클래스 변수는 프로그램이 실행되어서 종료될 때까지 유지되고, 프로그램 내의 모든 인스턴스와 클래스 변수가 정의된 인스턴스의 멤버에서 참조가 가능하기 때문에 공용변수라고도 함

클래스 멤버의 구분

변수의 구분

3 클래스 변수

- 프로그램 내에서 언제나 유지해야 할 데이터가 있다면 클래스변수로 선언해서 사용
- 클래스변수의 참조 방법은 “클래스이름.변수이름” 형식으로 참조

```
클래스이름.변수이름
참조변수.변수이름
```

클래스 멤버의 구분

변수의 구분

3 클래스 변수

```
package chapter05;

public class ClassVariable {

    static String name = "KING";        // 클래스 변수 name 선언
    static int age = 10;                // 클래스변수 age 선언

    void tellName() {
        System.out.println("나의 이름은 "+name+"입니다." ); // 멤버변수인 name을 참조한다.
    }
    void tellAge() {
        System.out.println("나는 "+age+"살 입니다.");   // 멤버변수인 age를 참조한다.
    }
```

클래스 멤버의 구분

변수의 구분

3 클래스 변수

```
    public static void main(String[] args) {
                                                                    // 인스턴스 생성 없이
        System.out.println("이름 : "+ClassVariable.name);  // 클래스이름.변수 형식으로 참조
        System.out.println("나이 : "+ClassVariable.age);  // 클래스이름.변수 형식으로 참조

        ClassVariable variable = new ClassVariable();
        System.out.println("이름 : "+variable.name);  // 참조변수.변수 형식으로 참조
        System.out.println("나이 : "+variable.age);  // 참조변수.변수 형식으로 참조
        variable.tellName();  // 인스턴스 내부의 멤버로 인식하기 때문에 메서드 호출에서도 변수를
                                 참조해서 출력
        variable.tellAge();
    }
}
```

클래스 멤버의 구분

메소드의 구분

인스턴스 메소드

클래스 내부에서 정의하는 메소드로 클래스의 멤버인 인스턴스 변수와 클래스 변수들을 사용

클래스의 목적에 맞게 클래스 내부에서 정의된 변수나 메소드와 관련한 처리

참조하는 방식은 “참조변수,메소드이름()” 형식으로 메소드를 호출

```
참조변수.메소드이름();
```

클래스 멤버의 구분

메소드의 구분

인스턴스 메소드

```
package chapter05;

public class InstanceMethod {

        String name = "KING"; //인스턴스 변수 선언

        // 인스턴스 메소드 정의
        void hi() {
                System.out.println("안녕하세요~!! 저는 "+name+"입니다.");
        }

        public static void main(String[] args) {
                InstanceMethod instanceMethod = new InstanceMethod();
                instanceMethod.hi(); // 참조변수를 통해 인스턴스 메소드 호출
        }
}
```

클래스 멤버의 구분

메소드의 구분

클래스 메소드

클래스 메소드도 클래스 변수와 동일하게 선언부 앞에 static 키워드를 써 정의

함수의 호출이나 사용 방법도 클래스 변수와 동일

클래스 메소드를 정의하는 위치는 인스턴스 메소드와 같이 클래스 내부에 정의하고
클래스 내부에서는 인스턴스의 멤버처럼 사용

클래스 멤버의 구분

메소드의 구분

클래스 메소드

외부에서 호출방법은 "클래스이름.메소드이름()" 또는 "참조변수.메소드이름()" 형식으로 호출

클래스 메소드의 호출은 클래스 변수와 마찬가지로 클래스 코드가 메모리에 로드가 되는 시점에 메소드가 생성되기 때문에 클래스 변수나 클래스 메소드 안에서만 호출이 가능

! 클래스 메소드 처리 블록 내부에서는 인스턴스 멤버를 쓸 수 없음

→ 이유는 클래스 멤버와 인스턴스 멤버의 생성 시점과 생명주기가 틀리기 때문에 사용할 수 없음

클래스 멤버의 구분

메소드의 구분

클래스 메소드

```
package chapter05;

public class ClassMethod {

        static String name = "KING"; // 클래스 변수 선언

        // 클래스 메소드의 정의
        static void hi() {
                System.out.println("안녕하세요~!! 저는 " + name + "입니다.");
        }

        public static void main(String[] args) {
                ClassMethod.hi(); // 클래스 메소드 호출
        }
}
```

생성자

COMPILER DESIGN
Reinhard Wilhelm · Dieter Maurer
COMPILER DESIGN
Java
C++

생성자

생성자는 인스턴스가 생성될 때마다 호출되는 초기화 메소드

클래스는 반드시 하나 이상의 생성자를 가지고 있어야 함

초기화 메소드	생성자는 클래스를 기반으로 인스턴스가 만들어질 때 단 한번 실행되는데 문법적인 구조가 메소드의 구성과 비슷함 인스턴스 생성 시 한번 처리되므로 인스턴스가 가지고 있는 변수를 초기화하는데 사용

생성자

생성자의 이름은
클래스의 이름을 씀

초기화에 필요한 데이터를 받기
위해 매개 변수를 정의

```
클래스의 이름  (타입 변수명, 타입 변수명, …) {
        // 인스턴스 생성 시 수행될 코드
        // 주로 인스턴스 변수의 초기화 코드를 적음
}
```

인스턴스 생성 시에 필요한
초기화 작업 코드를 작성

생성자

생성자의 구조

생성자의 이름과 초기화에 필요한 데이터를 받기 위한 매개변수, 처리를 위한 블록으로 이루어진 구조

➔ 메소드 선언과 비슷함

생성자 정의를 위해 충족되어야 할 조건

- 생성자의 이름은 클래스 이름을 사용함
- 메소드의 구조와 비슷하지만 반환형 타입을 쓰지 않음
- 매개변수의 정의는 메소드와 동일하게 정의함

생성자

생성자의 호출

객체가 생성되고 난 후 생성자를 호출

인스턴스가 생성되는 절차

```
SmartPhone sp = new SmartPhone();
```

① SmartPhone sp: 참조변수 선언

② new SmartPhone: 인스턴스 생성

③ SmartPhone(): 생성자 호출

참조변수가 선언이 되고, 다음으로 new SmartPhone 구문이 실행되어 클래스의 멤버들을 메모리에 올리는 인스턴스화가 진행

그 다음 생성자의 호출 과정이 진행되는데 SmartPhone() 생성자를 호출

기본 생성자(Default Constructor)

JAVA에서는 프로그래머가 클래스를 정의할 때 생성자를 정의하지 않아도
"기본 생성자" 라는 것을 자동으로 추가해서 프로그래머가 생성자를 정의하지 않고 생략할 수 있음

기본 생성자는 생성자의 매개변수가 정의되지 않고 처리블록에는 처리하는 문장이 없는 생성자를 의미

➜ 클래스 구성 시 생성자를 정의하지 않아도 JAVA에서 기본 생성자를 자동으로 추가하여 실행하도록 하기 때문에 생략이 가능

생성자에 아무것도 처리하지 않는 경우에는 생성자의 정의를 생략해도 되는 것

```
SmartPhone(){
}
```

매개변수를 받는 생성자

메소드의 정의와 같이 생성자의 이름 뒤 괄호"()" 안에 초기화에 필요한 데이터를 받을 수 있도록 매개변수를 정의해주는 생성자

매개변수로 필요한 데이터를 받아 초기화 작업에 사용

```
SmartPhone(int num){
        size = num;
}
```

매개변수를 받는 생성자

```
package chapter05.phone;

public class SmartPhone {

    String color; // 속성 : 스마트폰의 색상
    float size; // 속성 : 스마트폰의 사이즈
    int volume; // 속성 : 음량 크기

    //초기화에 필요한 데이터를 받기 위해 매개변수를 정의한 생성자
    public SmartPhone(String color, float size, int volume) { // 색상, 사이즈, 볼륨크기 값을 받
아 초기화
        this.color = color;
        this.size = size;
        this.volume = volume;
    }
```

매개변수를 받는 생성자

```
void call() { // 기능 : 전화걸기
    System.out.println("전화걸기");
}

void volumeUp() { // 기능 : 음량 키우기
    volume += 1;
}

void volumeDown() { // 기능 : 음량 낮추기
    volume -= 1;
}
```

매개변수를 받는 생성자

```
    public static void main(String[] args) {

        SmartPhone sp = new SmartPhone("RED", 10.5F, 0);
        System.out.println("> color : " + sp.color);      // 생성자를 통해 초기화된 color변수 참조
        System.out.println("> size : " + sp.size);         // 생성자를 통해 초기화된 size변수 참조
        System.out.println("> volume : " + sp.volume); // 생성자를 통해 초기화된  volume변수
참조
        sp.call(); // 참조변수를 이용해 객체의 메서드 호출

    }
}
```

여러 개의 생성자 정의

생성자도 메소드와 같이 오버로딩이 가능해서 여러 개의 생성자를 정의할 수 있음

생성자의 매개 변수의 타입이나 매개변수의 개수를 다르게 하여 생성자 여러 개를 정의할 수 있음

여러 개의 생성자 정의

```
package chapter05.phone;

public class SmartPhone2 {

    String color; // 속성 : 스마트폰의 색상
    float size; // 속성 : 스마트폰의 사이즈
    int volume; // 속성 : 음량 크기

    // 초기화에 필요한 데이터를 받기 위해 매개변수를 정의한 생성자
    public SmartPhone2(String color, float size, int volume) { // 색상, 사이즈, 볼륨크기 값을 받아 초기화
        this.color = color;
        this.size = size;
        this.volume = volume;
    }

    public SmartPhone2(String color, float size) { // 색상, 사이즈 값을 받아 초기화
        this.color = color;
        this.size = size;
        this.volume = 0;
    }
```

여러 개의 생성자 정의

```
void call() { // 기능 : 전화걸기
    System.out.println("전화걸기");
}

void volumeUp() { // 기능 : 음량 키우기
    volume += 1;
}

void volumeDown() { // 기능 : 음량 낮추기
    volume -= 1;
}

void printSpec() {
    System.out.println("> 스마트폰의 색상 : " + color);
    System.out.println("> 스마트폰의 사이즈 : " + size);
    System.out.println("> 스마트폰의 볼륨 크기 : " + volume);
}
```

여러 개의 생성자 정의

```
    public static void main(String[] args) {
        SmartPhone2 sp = new SmartPhone2("RED", 10.5F, 0);
        sp.printSpec();
        SmartPhone2 sp1 = new SmartPhone2("GOLD", 4.1F);
        sp1.printSpec();
    }
}
```

생성자를 호출하는 this()와 자기 자신을 가리키는 참조변수 this

생성자를 정의할 때 이미 정의한 다른 생성자를 호출

this()를 사용하는데 매개변수인자를 전달하는 형식으로 다른 생성자를 호출

메소드 호출과 비슷하게 매개변수의 인자의 개수나 타입으로 이미 정의된 생성자를 호출

참조 변수 this는 프로그래머가 정의 하지 않지만 인스턴스 생성과 동시에 생성되는 참조 변수

this를 이용해서 같은 이름의 지역변수와 인스턴스 변수를 구별해서 사용하거나 참조변수를 매개변수의 인자로 전달할 때 사용

생성자를 호출하는 this()와 자기 자신을 가리키는 참조변수 this

```
package chapter05.phone;

public class SmartPhone3 {

        String color; // 속성 : 스마트폰의 색상
        float size; // 속성 : 스마트폰의 사이즈
        int volume; // 속성 : 음량 크기

        //초기화에 필요한 데이터를 받기 위해 매개변수를 정의한 생성자
        public SmartPhone3(String color, float size, int volume) { // 색상, 사이즈, 볼륨크기 값을 받아
                                                                    초기화
                this.color = color;
                this.size = size;
                this.volume = volume;
        }
```

생성자를 호출하는 this()와 자기 자신을 가리키는 참조변수 this

```
public SmartPhone3(String color, float size) { // 색상, 사이즈 값을 받아 초기화
        this(color, size, 0);
}

public SmartPhone3(String color) { // 색상 값을 받아 초기화
        this(color, 5.7f, 0);
}

public SmartPhone3(float size) { // 사이즈 값을 받아 초기화
        this("BLACK", size, 0);
}
```

생성자를 호출하는 this()와 자기 자신을 가리키는 참조변수 this

```
void call() { // 기능 : 전화걸기
        System.out.println("전화걸기");
}

void volumeUp() { // 기능 : 음량 키우기
        volume += 1;
}

void volumeDown() { // 기능 : 음량 낮추기
        volume -= 1;
}

void printSpec() {
        System.out.println("> 스마트폰의 색상 : " + color);
        System.out.println("> 스마트폰의 사이즈 : " + size);
        System.out.println("> 스마트폰의 볼륨 크기 : " + volume);
}
```

생성자를 호출하는 this()와 자기 자신을 가리키는 참조변수 this

```
        public static void main(String[] args) {

                SmartPhone3 sp = new SmartPhone3("RED", 10.5F, 0);
                sp.printSpec();
                System.out.println();
                SmartPhone3 sp1 = new SmartPhone3("GOLD", 4.1F);
                sp1.printSpec();
                System.out.println();
                SmartPhone3 sp2 = new SmartPhone3("BLUE");
                sp2.printSpec();
                System.out.println();
                SmartPhone3 sp3 = new SmartPhone3(5.0F);
                sp3.printSpec();
        }
}
```

패키지와 제어자

COMPILER DESIGN
Reinhard Wilhelm • Dieter Maurer
COMPILER DESIGN
Java
C++

package와 import

JAVA에서 패키지는 서로 관련된 클래스와 인터페이스의 묶음

패키지 안에는 클래스와 인터페이스 그리고 또 다른 패키지를 포함할 수 있음

패키지를 사용함으로써 서로 관련된 클래스와 인터페이스를 한대 묶어 효율적인 관리가 가능

패키지를 나눔으로써 같은 이름의 클래스나 인터페이스의 충돌을 방지할 수 있음

패키지 구조의 접근은 dot(.)으로 구분해서 사용

```
java.until.Date
```

Date 클래스는 java 패키지 안에 존재하는 util 패키지 안에 존재하는 클래스

package와 import

패키지 선언 방법

package 패키지 이름;

현재 작성중인 클래스의 가장 위에 위치한 패키지 선언문

chapter05 패키지 안의 phone 패키지임을 의미

```
package chapter05.phone;
```

package와 import

import 구문의 작성 방법

import 패키지이름.클래스이름;
또는
import 패키지이름.*;

패키지나 클래스를 import 하면 클래스의 이름만으로도 사용이 가능해짐

1. 클래스 파일에서 패키지의 선언의 위치와 import 문의 위치는 패키지 선언
2. import 문
3. 클래스 선언

```
package 패키지 이름;
```

```
import 패키지이름.클래스이름;
import 패키지이름.*;
```

```
class 클래스이름{
}
```

package와 import

```
package chapter05; //현재 클래스의 패키지

import java.util.Date; // java.util.Date 클래스를 import

public class ImportDate {

    public static void main(String[] args) {

        java.util.Date now1 = new java.util.Date(); // Date 클래스의 기본 이름은 java.util.Date 입니다.
        Date now2 = new Date(); // Date 클래스를 import 해서 간단하게 Date로 쓸 수 있다.
        System.out.println(now1);
        System.out.println(now2);
    }
}
```

접근제어자

접근제어자는 외부의 다른 인스턴스에서 인스턴스 멤버들을 참조하거나 호출할 때 멤버들을 숨기는 정보은닉이 가능하도록 해주는 제어자

클래스, 변수, 메소드 등을 정의할 때 제어자를 사용하여 다른 인스턴스에서의 접근을 제어할 수 있음

JAVA에서 제공하는 접근제어자는 4가지가 있고 접근할 수 있는 서로 다른 범위를 가짐

접근제어자

1. private: 클래스 내부의 멤버들만 접근 가능
2. default: 같은 클래스 내부와 같은 패키지의 인스턴스까지 접근 가능
3. protected: 같은 클래스, 같은 패키지와 상속 관계의 하위 클래스에서 접근 가능
4. public: 제약이 없이 어디서든 접근이 가능

접근제어자

접근 한정자	동일한 클래스	같은 패키지 내의 모든 클래스 (상속 여부에 상관없음)	다른 패키지	
			상속된 클래스	상속되지 않은 클래스
public	O	O	O	O
protected	O	O	O	X
default	O	O	X	X
private	O	X	X	X

접근제어자

private

```
package chapter05;

public class ModifierPrivate {

	private String name; // 클래스의 인스턴스 변수는 내부에서만 접근 가능
	private int age;

	void tell() {
		System.out.println("안녕하세요. " + age + "살 " + name + "입니다.");
	}
}
```

접근제어자

private

```
package chapter05;

public class ModifierPrivateMain {

	public static void main(String[] args) {

		ModifierPrivate mp = new ModifierPrivate();
		mp.name = "손흥민";   // 현재 클래스에서는 인스턴스 변수에 접근이 불가
		mp.tell();

	}
}
```

접근제어자

default

```
package chapter05;

public class ModifierDefault {

	String name; //클래스의 인스턴스 변수는 같은 패키지 내부에서만 접근 가능
	int age;

	void tell() {
		System.out.println("안녕하세요. " + age + "살 "+ name +"입니다.");
	}

}
```

접근제어자

default

```
package chapter05;

import chapter05.ModifierDefault;

public class ModifierDefaultMain {

    public static void main(String[] args) {

        ModifierDefault mp = new ModifierDefault();

        mp.name = "손흥민"; // 현재 클래스에서는 같은 패키지에서 접근하므로 인스턴스 변수에 접근 가능
        mp.age = 28; // 현재 클래스에서는 같은 패키지에서 접근하므로 인스턴스 변수에 접근 가능

        mp.tell(); // 현재 클래스에서는 같은 패키지에서 접근하므로 인스턴스 변수에 접근이 가능

    }
}
```

접근제어자

default

```
package chapter05.other; //사용하는 인스턴스와 다른 패키지

import chapter05.ModifierDefault;

public class ModifierDefaultMain {

    public static void main(String[] args) {
        ModifierDefault mp = new ModifierDefault();
        mp.name = "손흥민";   // 현재 클래스에서는 다른 패키지에서 접근하므로 인스턴스 변수에 접근이 불가
        mp.age = 28;    // 현재 클래스에서는 다른 패키지에서 접근하므로 인스턴스 변수에 접근이 불가
        mp.tell();   // 현재 클래스에서는 다른 패키지에서 접근하므로 인스턴스 변수에 접근이 불가
    }
}
```

접근제어자

protected

```
package chapter05;

public class ModifierProtected {

	protected String name;
	int age;

	protected void tell() {
		System.out.println("안녕하세요. " + age + "살 "+ name +"입니다.");
	}
}
```

접근제어자

protected

```
package chapter05.other; //상속하는 클래스와 다른 패키지

import chapter05.ModifierProtected;

public class ModifierProtectedExtends extends ModifierProtected {

    public static void main(String[] args) {

        ModifierProtectedExtends mpe = new ModifierProtectedExtends();
        mpe.name = "손흥민"; // 접근제어자가 protected이기 때문에 상속 관계에서도 접근이 가능
        // mpe.age=28;   // 접근제어자가 default이기 때문에 상속 관계에서도 접근이 불가
        mpe.tell(); // 접근제어자가 protected이기 때문에 상속 관계에서도 접근이 가능

    }
}
```

접근제어자

public

```
package chapter05;

public class ModifierPublic {

	public String name;
	public int age;

	public void tell() {
		System.out.println("안녕하세요. " + age + "살 "+ name +"입니다.");
	}

}
```

접근제어자

public

```
package chapter05.other;   // 메인 메소드에서 생성한 인스턴스 다른 패키지

import chapter05.ModifierPublic;

public class ModifierPublicMain {

        public static void main(String[] args) {

                ModifierPublic mp = new ModifierPublic();
                mp.name = "손흥민";   //  접근이 가능
                mp.age = 28;    //   접근이 가능
                mp.tell();   //   접근이 가능

        }

}
```

데이터 보호와 캡슐화

캡슐화	인스턴스의 변수를 임의로 바꾸지 못하도록 제한하거나 특정 메소드를 호출하지 못하도록 제한하는 것

데이터의 참조나 메소드의 호출을 제한할 때는 접근제어자를 이용

- 클래스의 캡슐화를 위해 일반적으로 변수 앞에는 private를 사용해서 변수에 직접 참조하는 것을 막음
- public 메소드를 통해서만 참조하고 메소드를 통해서만 데이터가 변경이 되도록 함

변수의 값을 제공하는 getter메소드와 변수에 값을 설정하는 setter메소드를 사용

데이터 보호와 캡슐화

```
package chapter05;

public class Person {

	private String name; // 변수는 외부에서 직접 참조가 불가능하도록 private 적용
	private int age;

	// 변수에 값의 변경이나 반환을 위한 setter/getter 메소드는 public으로 정의해서 외부에서
	   접근이 가능하도록 정의
	public String getName() {
		return name;
	}

	public void setName(String name) {
		this.name = name;
	}
```

데이터 보호와 캡슐화

```
        public int getAge() {
                return age;
        }

        public void setAge(int age) {
                this.age = age;
        }
        public void tell() {
                System.out.println("안녕하세요. " + age + "살 " + name + "입니다.");
        }
}
```

데이터 보호와 캡슐화

```
package chapter05;

public class PersonMain {

	public static void main(String[] args) {

		Person person = new Person();

		// person.name = "손흥민"; // 변수에 직접 참보는 불가능
		person.setName("손흥민");
		person.setAge(28);
		person.tell();
	}
}
```